

JavaScript-03

AJAX- & Fetch-anrop

28 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin



AJAX- och Fetch-anrop

AJAX - Introduktion

AJAX är en webbutvecklingsteknik som gör det möjligt för en webbsida att kommunicera med en server, uppdatera sitt innehåll och göra det utan att orsaka en störande helomladdning av sidan. Det görs med JavaScript och XML (även om JSON används vanligare nuförtiden) för att skicka och ta emot data.

Den verkliga magin uppstår när JavaScript och XMLHttpRequest-objektet används för att utbyta data asynkront med en server. Det är härifrån 'A'et i AJAX kommer – asynkront. Det betyder att det är möjligt att skicka data till, och hämta data från, en server i bakgrunden, vilket gör att användaren kan interagera med sidan som vanligt medan AJAX-begäran bearbetas

Det finns ett JavaScript-bibliotek, jQuery, som ofta används tillsammans med AJAX för att göra syntaxen lite lättare. I den här genomgången visas AJAX-anrop utan hjälpbiblioteket jQuery. I slutet av det här häftet kommer en kort genomgång av hur AJAX-anrop ser ut med jQuery.

AJAX - Grunden

För att göra ett AJAX-anrop använder man ett XMLHttpRequest-objekt. Det finns i alla webbläsare. Om du öppnar en webbläsare och trycker på knappen **F12** så får du upp developer tools. Längst upp i den kan du välja en flik som heter **Console**. Om du efter symbolen > skriver in det som står i rutan för bild 1, så bör du få samma svar tillbaka.

```
> let xhr = new XMLHttpRequest();  
  
undefined  
  
> xhr  
  
XMLHttpRequest {onreadystatechange: null, readyState: 0, timeout: 0, withCredentials: false, upload: XMLHttpRequestUpload, ...}
```

Bild 1 Skapa en variabel xhr av typen XMLHttpRequest.

Det vi gjorde i developer tools var att skapa en variabel som heter xhr och är av datatypen XMLHttpRequest. Det är den som används för att göra AJAX-anrop.

Trots namnet XMLHttpRequest kan det användas för att hämta **alla** typer av data, inte bara XML.

Objektet XMLHttpRequest fungerar genom att först skapa en instans av objektet, konfigurera en begäran och sedan skicka begäran. När begäran har skickats definieras en funktion för att hantera svarsdata från servern. Här är en snabb översikt över hur ett AJAX-anrop XMLHttpRequest kan se ut:

```
let xhr = new XMLHttpRequest();

xhr.open('GET', 'https://api.example.com/data', true);
xhr.onreadystatechange = function() {
  if (xhr.readyState == 4 && xhr.status == 200)
    console.log(xhr.responseText);
}

xhr.send();
```

Bild 2 Exempel på ett GET-anrop med AJAX.

AJAX – Anrop – steg för steg

1. Skapa en instans av XMLHttpRequest

Det första steget i att göra ett AJAX-anrop utan jQuery är att skapa en instans av XMLHttpRequestobjektet. Detta objekt tillhandahåller metoder för att överföra data mellan en klient och en server.

```
let xhr = new XMLHttpRequest();
```

Bild 3 Börja med att skapa en instans av XMLHttpRequest.

2. Inställning av instansen

När vi har skapat en instans av XMLHttpRequest, är nästa steg att ordna med inställningar av (konfigurera) vår förfrågan. Detta görs med hjälp av open()metoden, som tar tre parametrar:

- typen av förfrågan (GET, POST, etc.),
- URL:en som förfrågan skickas till och
- om förfrågan ska göras asynkront.

```
xhr.open('GET', 'https://api.example.com/data', true);
```

Bild 4 Konfigurera instansen.

Här gör vi en GET-förfrågan asynkront till <https://api.example.com/data>. Den tredje parametern sätts till true för att göra förfrågan asynkron. Om du ställer

in den till false kommer förfrågan att vara synkron och körningen av JavaScript-koden pausas tills servern svarar.

3. Hantering av svaret

Innan begäran faktiskt kan skickas måste vi ange hur svaret från servern ska hanteras. Detta görs med hjälp av `onreadystatechange`, `eventlistener`, som körs varje gång `readyState`objektets egenskap ändras.

```
xhr.onreadystatechange = function() {  
    if (xhr.readyState == 4 && xhr.status == 200) {  
        console.log(xhr.responseText);  
    }  
}
```

Bild 5 Inställning av eventlistener.

I den här koden kontrollerar vi om `readyState` är 4 och om statuskoden är 200. Om båda villkoren är true loggar vi ut svarstexten till konsolen.

Egenskapen `status` returnerar statuskoden för en begäran. 200 betyder "OK". Andra vanliga statuskoder inkluderar 404 (Not Found) och 500 (Server Error).

4. Skicka http-request

Nu när `request`-objektet är konfigurerat skickar man iväg den med metoden `send()`.

```
xhr.send();
```

Bild 6 Skicka iväg request.

Den här kodraden skickar `request` till servern. Om du gör en POST-request skickar du dina data som en parameter till `send()` metoden.

AJAX – Felhantering

Precis som inom all programmering kommer olika fel att uppstå. Man behöver hantera dessa fel för att undvika problem med din applikation eller webbsida. När det gäller AJAX-anrop kan man hantera fel med hjälp av `XMLHttpRequest`-objektets eventlyssnare **onerror**.

```
let xhr = new XMLHttpRequest();

xhr.open('GET', 'https://api.github.com/users/stackabuse', true);

xhr.onerror = function() {
  console.log('Request Error...');
}

xhr.send();
```

Bild 7 Felhantering med hjälp av onerror.

I koden på bild 7 ovanför, har vi kopplat en funktion till onerror. Denna funktion anropas om ett fel uppstår med AJAX-förfrågan. I det här fallet loggar vi helt enkelt ett meddelande till konsolen, men i en verklig applikation kan du hantera det på några olika sätt:

- Informera användaren om felet
- Försök igen med begäran
- Misslyckas tyst

Naturligtvis finns det andra sätt att hantera dessa fel.

	Obs! Händelsehanteraren onerror utlöses inte för HTTP-fel som 404- eller 500-status. Den utlöses bara för fel på nätverksnivå, till exempel att begäran blockeras av en CORS-policy eller att användaren förlorar internetanslutningen. Http-status 404 => Sidan hittas inte Http-status 500 => Servern har internt fel
---	--

Bild 8 Information om Http-fel.

Det går att kontrollera vilken typ av fel man får i eventhandler onload().

```
var xhr = new XMLHttpRequest();

xhr.open('GET', 'https://api.github.com/users/stackabuse', true);

xhr.onload = function() {
  if (xhr.status >= 200 && xhr.status < 400) {
    // Success!
    console.log(xhr.responseText);
  } else {
    // We reached the server, but it returned an error
    console.log('Server Error!');
  }
};

xhr.onerror = function() {
  // There was a connection error of some sort
  console.log('Connection Error!');
};

xhr.send();
```

Bild 9 Hantering in onload-metoden för objektet.

I kodavsnittet i bild 9 kontrollerar vi om statuskoden för response ligger i intervallet 200–399, vilket representerar lyckade HTTP-svar. Om den inte ligger inom detta intervall loggar vi ett felmeddelande.

Fetch - Introduktion

Fetch API används för att göra HTTP-requests, liknande XMLHttpRequest. Till skillnad från XMLHttpRequest behöver man inte skapa ett nytt objekt för att använda Fetch API.

Webbläsare har en global fetch()-funktion som man kan använda för att göra requests.

Man kan bland annat göra **GET**, **POST**, **PUT**, **PATCH**, **DELETE**, men även andra typer av Http-requests. Vi börjar med att titta en GET-request.

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Bild 10 Exempel på ett GET-anrop med fetch med hantering av promises.

Anropet med fetch() returnerar något som man kallar för **promise**. När skickar iväg anropet med fetch() så börjar fetch()-funktionen leta efter data på en gång. Men den har inte datat när

anropet görs och den vet inte heller om det kommer någon data tillbaka alls. Det kan ju bli något fel på vägen. Så därför returnerar funktionen något som kallas för promise istället. Det är ungefär som ett framtida löfte om att, "hittar jag någon data, så lovar jag att **returnera** den till dig, men om jag inte hittar någon data, så lovar jag att du ska få **en anledning** till varför jag inte lyckades hitta någonting".

```
// returns a promise, does not store data
const characters = fetch('https://swapi.dev/api/people');
```

Bild 11 Exempel på ett GET-anrop med fetch.

När anropet har gått iväg får man helt enkelt sitta och vänta tills ett promise kommer tillbaka, då kan man börja hantera och agera utifrån det. Det finns metoder för att hantera olika lägen, eller tillstånd (eng. state) i promise. Det är `.then()` och `.catch()`.

Det finns även andra metoder att tillgå, men det är de här två som är viktigast. När promise först kommer tillbaka och man inte vet om något vettigt kommer tillbaka eller om det blir någon form av error, så är promise i ett läge som kallas **pending**. När det ändras till nästa läge så kan promise antingen vara **fulfilled** eller **rejected**. Fulfilled innebär att det har gått bra och promise är redo att leverera det data som du har väntat så länge på. Rejected blir det om något har gått fel och du får då ta emot någon form av felmeddelande istället.

then()	Den här körs endast om promise har kommit till läget fulfilled, det vill säga att allt har gått bra och det finns data som har kommit tillbaka.
catch()	Den här körs endast om promise har läget rejected, alltså att det inte gick att hämta data och det kommer ett felmeddelande istället.

Bild 12 Förklaring av then() och catch()

Fetch – Anrop – steg för steg

När man använder sig av `then()` och `catch()` så säger man att man "kedjar på" (eng. chain) metoderna efter fetch. Det är vanligt i JavaScript.

```
const tecken = fetch('https://swapi.dev/api/people');

tecken
  .then (data => data.json())
  .then (data => {
    console.log(data)
  })
  .catch((error) => {
    console.error(error)
  })
```

Bild 13 Exempel på chain av then och catch.

På bild 13 ser koden lite konstig ut eftersom det börjar med `.then` och `.catch` på nya rader, men det är för läsbarhetens skull. Samma kod skulle kunna skrivas enligt bild 14 också.

```
const tecken = fetch('https://swapi.dev/api/people');

tecken.then(data => data.json()).then(data => {
  console.log(data)
}).catch((error) => {
  console.error(error)
})
```

Bild 14 Exempel på chain av `then` och `catch`, lite ihoptryckt.

Båda sätten att skriva på är fullt godkända och förekommer, men det första enligt bild 13 är att föredra, och det är också det vanligaste sättet att skriva på. Vi kollar en gång till på koden i bild 15.

```
// returnerar ett promise
const tecken = fetch('https://swapi.dev/api/people');

// chain-metoder på promise
tecken
// promise fulfilled - anropa en metod med svaret
  .then(data => data.json())
// har data i det format vi vill ha, nu console.log data
  .then(data => {
    console.log(data)
  })
  .catch((error) => {
    console.error(error)
  })
```

Bild 15 Exempel på chain av `then` och `catch` med några kommentarer.

På översta raden är variabeln `tecken` satt till resultatet av `fetch`-anropet, ett `promise` alltså. På raderna under finns det längst ner en `catch`-metod dit koden går om läget på `promise` råkar bli `rejected`. Det enda som händer då är att den struntar i båda `then()`-metoderna och hoppar rakt ner till `catch()`. Där körs bara en vanlig `log` i konsolen.

Om läget på `promise` istället blir `fulfilled` så går körningen av koden in i den första `then()`-metoden, med `data` som den levererar tillbaka. Den här datan är i ett format som inte går att använda, därför kör man `data.json()` för att få det till `json`-format istället. Den hanteringen, `data.json()` returnerar också ett `promise`. När det är klart går körningen in i den andra `then()`-metoden där man också gör en vanlig `log` i konsolen.

AJAX – med hjälp av jQuery

jQuery är ett JavaScript-bibliotek som underlättar själva skrivandet av några krångliga JavaScript-kommandon. För att kunna använda det behöver man lägga till biblioteket i sin index.html enligt bild 16.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AJAX Example</title>
  <!-- Include jQuery -->
  <script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
</head>
```

Bild 16 Inkludera jQuery i filen index.html.

Nu kan man skriva i sin JavaScript-kod på följande sätt om man vill göra ett anrop till en endpoint.

```
$.ajax({
  url: 'https://api.example.com/data',
  method: 'POST',
  data: { key1: 'value1', key2: 'value2' },
  dataType: 'json',
  success: function(data) {
    console.log('Data received:', data);
  },
  error: function(error) {
    console.error('Error:', error);
  }
});
```

Bild 17 Ett AJAX-anrop med ett objekt.

När man använder jQuery första gången så ser det lite konstigt ut med dollartecken, men det är skrivsättet för att komma åt biblioteket jQuery. Sen finns det massor av metoder som man kan använda, en av dessa är metoden .ajax(). Den tar ett objekt med flera attribut. Några av de viktigaste attributen ser vi i bild 17, url, method, dataType, success och error.

url	Den endpoint-adress som man vill nå.
method	Den HTTP-request-metoden man vill använda.
data	Den typen av data man vill skicka
dataType	Vilken datatyp förväntas man få tillbaka från servern
success	Den funktion man vill köra om allt går bra
error	Den funktion man vill köra om allt skiter sig

Bild 17 Ett AJAX-anrop med ett objekt.